

```
In [1]: #include <iostream>
#include <fstream>
#include <vector>
#include <TGraph.h>
#include <TCanvas.h>
#include <string>
#include <cmath>
#include <array>
#include <algorithm>
```

The U.S. Standard Atmosphere, 1976 model computes atmospheric properties (pressure, density, temperature) based on altitude (h), defined from sea level up to 1000 km. It is primarily determined by a defined temperature profile (T vs h), using the hydrostatic equation and ideal gas law to calculate pressure and density across different layers. [1, 2, 3, 4, 5]

Key Sea Level Constants (0 km)

- Pressure (): 101325 Pa ($1.01325 \times 10^5 \text{ N/m}^2$).
- Temperature (): 288.15 K (15 °C or 59 °F).
- Density (): 1.2250 kg/m³.
- Gravitational Acceleration (): 9.80665 m/s². [2, 6]

Core Equations (Troposphere to Stratosphere) The model splits the atmosphere into layers with specific temperature gradients ($L_m = \frac{dT_m}{dh}$). [1, 4]

1. Temperature () at Geopotential Altitude (): $T_m = T_b + L_m(h - h_b)$ Where T_b is the reference temperature at layer bottom, and L_m is the lapse rate.

2. Pressure () Calculation:

- If lapse rate : $P = P_b \left[\frac{T_b}{T_b + L_m(h - h_b)} \right]^{\frac{g_0 M}{R^* L_m}}$

- If lapse rate (Isothermal): $P = P_b \exp \left[\frac{-g_0 M(h - h_b)}{R^* T_b} \right]$

3. Density () Calculation: $\rho = \frac{PM}{R^* T_m}$

- : Molecular weight of air (28.9644 kg/kmol).
- : Universal gas constant (8314.32 J/(kmol · K)).
- : Geopotential altitude at the base of the layer. [4, 6, 8, 9, 10]

Key Layers (0 to ~86 km) [2]

The 1976 model provides specific gradients (L_m) for calculating T and P within these layers:

- Troposphere (): Temperature decreases by 6.5 K/km.
- Lower Stratosphere (): Temperature is constant (216.65 K).
- Upper Stratosphere (): Various gradients define increasing and then decreasing temperature up to the mesopause. [2, 4, 11, 12, 13]

Note: The 1976 Standard Atmosphere is generally identical to ICAO Standard (1964) up to and ISO Standard (1973) to . [1]

```
In [4]: double dt_s = 0.1;
double t_max_s = 400.0; // Run long enough to see all stages
int n_points = static_cast<int>(t_max_s / dt_s);

// Initial LVM3 mass (640 Tonnes)
double current_mass_kg = 640000.0;

// Outputs from the function
double mass_flow_rate_kg_s, exit_velocity_m_s, exit_pa, nozzle_area_m2;

std::vector<double> time_axis_s;
std::vector<double> mass_axis_kg;

// — 2. Simulation Loop —
for (int i = 0; i < n_points; ++i) {
    double current_t_s = i * dt_s;

    // We simulate a fake altitude that increases by 500m every second
    // to trigger the staging heights (43km, 60km, 175km)
    double simulated_altitude_m = 500.0 * current_t_s;

    // Call the library function
    get_lvm3_stage_properties(simulated_altitude_m, dt_s, current_mass_kg,
                             mass_flow_rate_kg_s, exit_velocity_m_s,
                             exit_pa, nozzle_area_m2);

    // Record telemetry
    time_axis_s.push_back(current_t_s);
    mass_axis_kg.push_back(current_mass_kg);

    // Safety break if mass somehow goes negative (bug detection)
    if (current_mass_kg < 0) break;
}

// — 3. ROOT Plotting —
auto canvas_mass = new TCanvas("c_mass", "Mass Expulsion Profile", 800, 600)

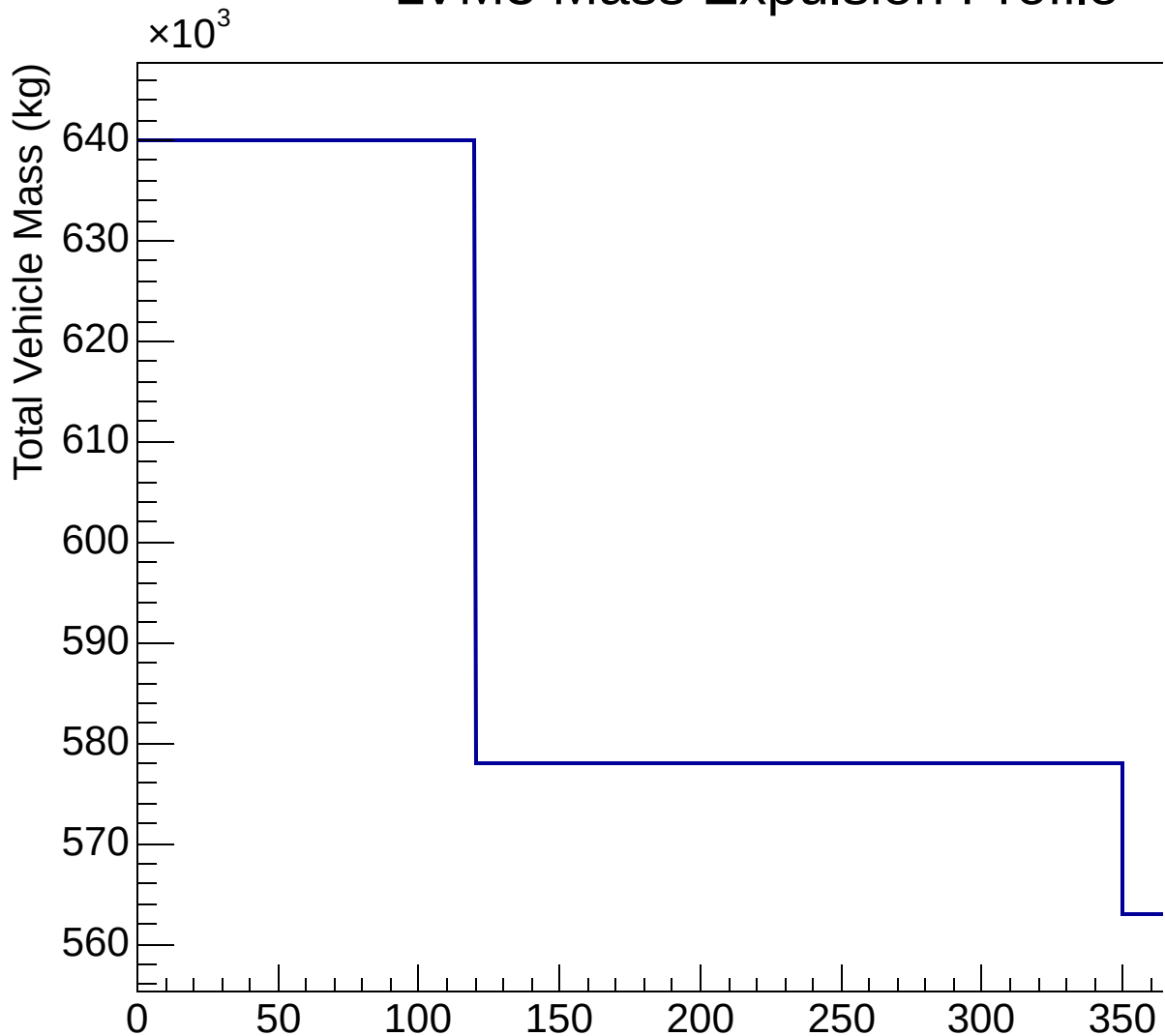
// Correct Order: TGraph(n, x_array, y_array) -> (time, mass)
auto graph_mass = new TGraph(time_axis_s.size(), time_axis_s.data(), mass_ax

graph_mass->SetTitle("LVM3 Mass Expulsion Profile;Time (s);Total Vehicle Mas
graph_mass->SetLineColor(kBlue + 2);
graph_mass->SetLineWidth(2);

graph_mass->Draw("AL"); // 'A' for axes, 'L' for line
canvas_mass->Draw();

printf("Simulation Complete. Final Mass: %.2f kg\n", mass_axis_kg.back());
```

LVM3 Mass Expulsion Profile



Simulation Complete. Final Mass: 563000.00 kg

```
In [6]: auto x = 0.53
```

```
In [5]: auto y = 0.72
```

```
(double) 0.72000000
```

```
In [7]: for(int i = 0; i < 9; i++) {  
        x *= 1.001;  
    }  
x
```

```
(double) 0.53478912
```

```
In [9]: printf("result is %f \n", x + y)
```

```
result is 1.254789
```

```
(int) 20
```

```
In [23]: acos(sqrt(3)/2) // pi/6 rad
```

```
(double) 0.52359878
```

```
In [11]: sqrt(x)
```

```
(double) 0.73129278
```

The TMath namespace in the [ROOT data analysis framework](#) provides a comprehensive set of free functions for numerical, statistical, and geometrical calculations. While many functions are optimized wrappers for the standard C++ library, others offer specialized algorithms or increased precision. [1, 2]

Core Categories of TMath Functions## 1. Elementary Math and Trigonometry

These cover basic arithmetic operations and trigonometric functions. They are often used within TF1 (1D function) or [TFormula](#) definitions. [1, 3, 4]

- Elementary: TMath::Sqrt(x), TMath::Exp(x), TMath::Log(x), TMath::Power(x, y), TMath::Abs(x).
- Trigonometric: TMath::Sin(x), TMath::Cos(x), TMath::Tan(x), and their inverses (ASin, ACos, ATan).
- Hyperbolic: TMath::SinH(x), TMath::CosH(x), TMath::TanH(x). [5, 6, 7, 8, 9]

2. Statistical Distributions

TMath includes many [predefined probability density functions \(PDFs\)](#) and cumulative distributions. [1, 10]

- Common Distributions: TMath::Gaus(x, mean, sigma), TMath::Poisson(x, par), TMath::Landau(x, mpv, sigma), TMath::Binomial(n, k).
- Statistical Tests: TMath::Prob(chi2, ndf) and TMath::KolmogorovProb(z).
- Quantiles: TMath::NormQuantile(p), TMath::ChisquareQuantile(p, ndf). [5, 7, 11, 12, 13]

3. Algorithms for Arrays

These functions are designed to operate on C-style arrays or C++ iterators. [1, 2, 14]

- Statistics: TMath::Mean(), TMath::Median(), TMath::RMS(), TMath::StdDev().
- Searching & Sorting: TMath::Sort(), TMath::BinarySearch(), TMath::LocMin(), TMath::LocMax(). [1, 2, 7, 15]

4. Special Functions

These are advanced mathematical routines often used in high-energy physics. [1, 2, 10]

- Bessel Functions: `TMath::BesselJ0(x)`, `TMath::BesselI(n, x)`, `TMath::BesselK(n, x)`.
- Error & Gamma Functions: `TMath::Erf(x)`, `TMath::Erfc(x)`, `TMath::Gamma(z)`, `TMath::LnGamma(z)`.
- ROOT-Specific Specials: `TMath::DiLog(x)` and `TMath::StruveH0(x)` (not typically found in standard C++ libraries). [1, 7, 11, 15, 16]

Numerical Constants

TMath also provides physical and mathematical constants as inline functions rather than macros: [2, 16]

- `TMath::Pi()`, `TMath::E()`, `TMath::C()` (Speed of Light).
- `TMath::K()` (Boltzmann constant), `TMath::G()` (Gravitational constant), `TMath::H()` (Planck's constant). [5, 7, 11]

Usage Notes

- Modern Preference: For many special functions and distributions, ROOT now encourages using the newer `ROOT::Math` namespace (part of MathCore) for better performance and alignment with modern standards.
- Headers: To use these functions in a standalone C++ script, you must include the [TMath header](#):

```
include "TMath.h"double val =
TMath::Gaus(x, 0, 1);
```

[1, 2, 12, 16]

[1] <https://root.cern> [2] <https://root.cern.ch> [3] <https://root.cern.ch> [4] <https://root.cern>
 [5] <https://root.cern> [6] <https://www.fuw.edu.pl> [7] <https://github.com> [8]
<https://root.cern> [9] <https://www.scaler.com> [10] <https://root.cern> [11]
<https://root.cern.ch> [12] <https://root-forum.cern.ch> [13] <https://root.cern> [14]
<https://root-forum.cern.ch> [15] <https://root.cern.ch> [16] <https://root.cern>

```
In [15]: double mu = 0.0; double sigma = 1.0;
```

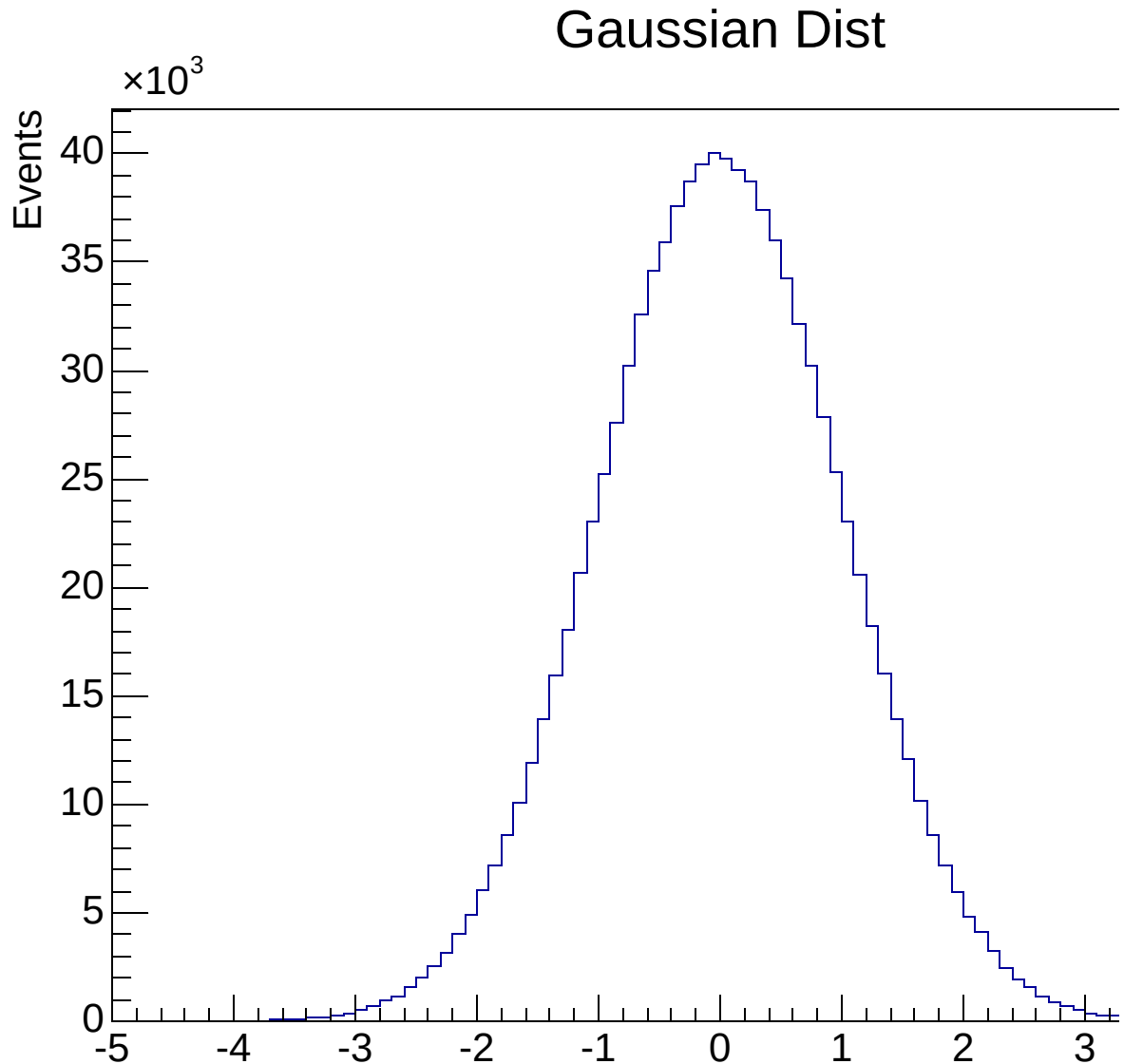
```
In [19]: auto random_from_gauss_distr= gRandom->Gaus(mu, sigma)
(double) 0.99893272
```

```
In [21]: TMath::Cos(3)
(double) 1.7320508
```

```
In [4]: %jsroot on
```

```
Info in <TCanvas::MakeDefCanvas>: created default TCanvas with name c1
```

```
In [9]: TCanvas *c1 = new TCanvas("c1", "Canvas", 800, 600);  
auto h2 = new TH1F("h2", "Gaussian Dist;X-axis;Events", 100, -5, 5);  
h2->FillRandom("gaus", 1000000);  
h2->Draw();  
c1->Draw();
```

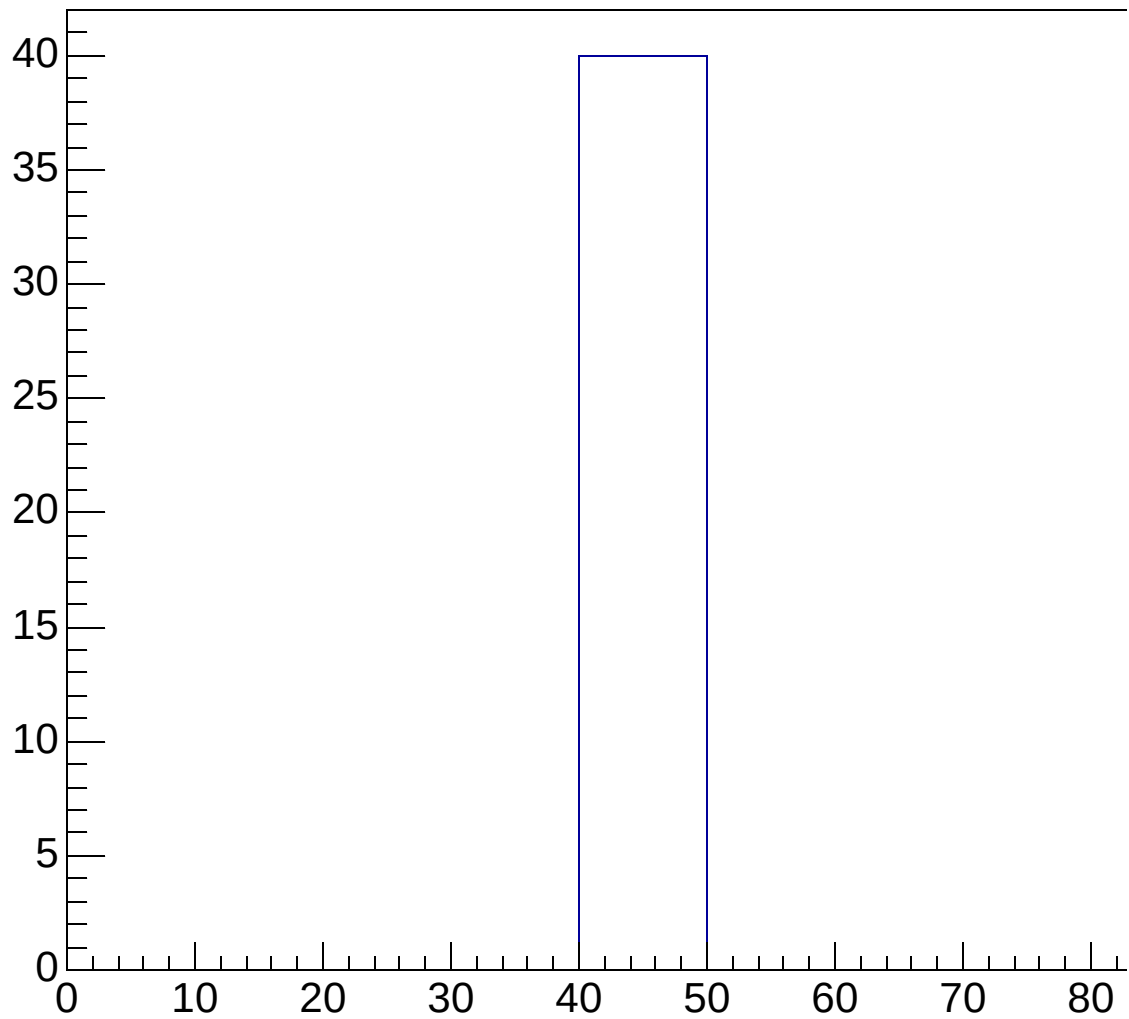


```
Warning in <TCanvas::Constructor>: Deleting canvas with same name: c1  
Warning in <TR00T::Append>: Replacing existing TH1: h2 (Potential memory leak).
```

```
In [6]: auto h1 = new TH1I("histname", "Histogram Title", 10, 0., 100.);  
h1->SetBinContent(5, 40);  
h1->Draw();
```

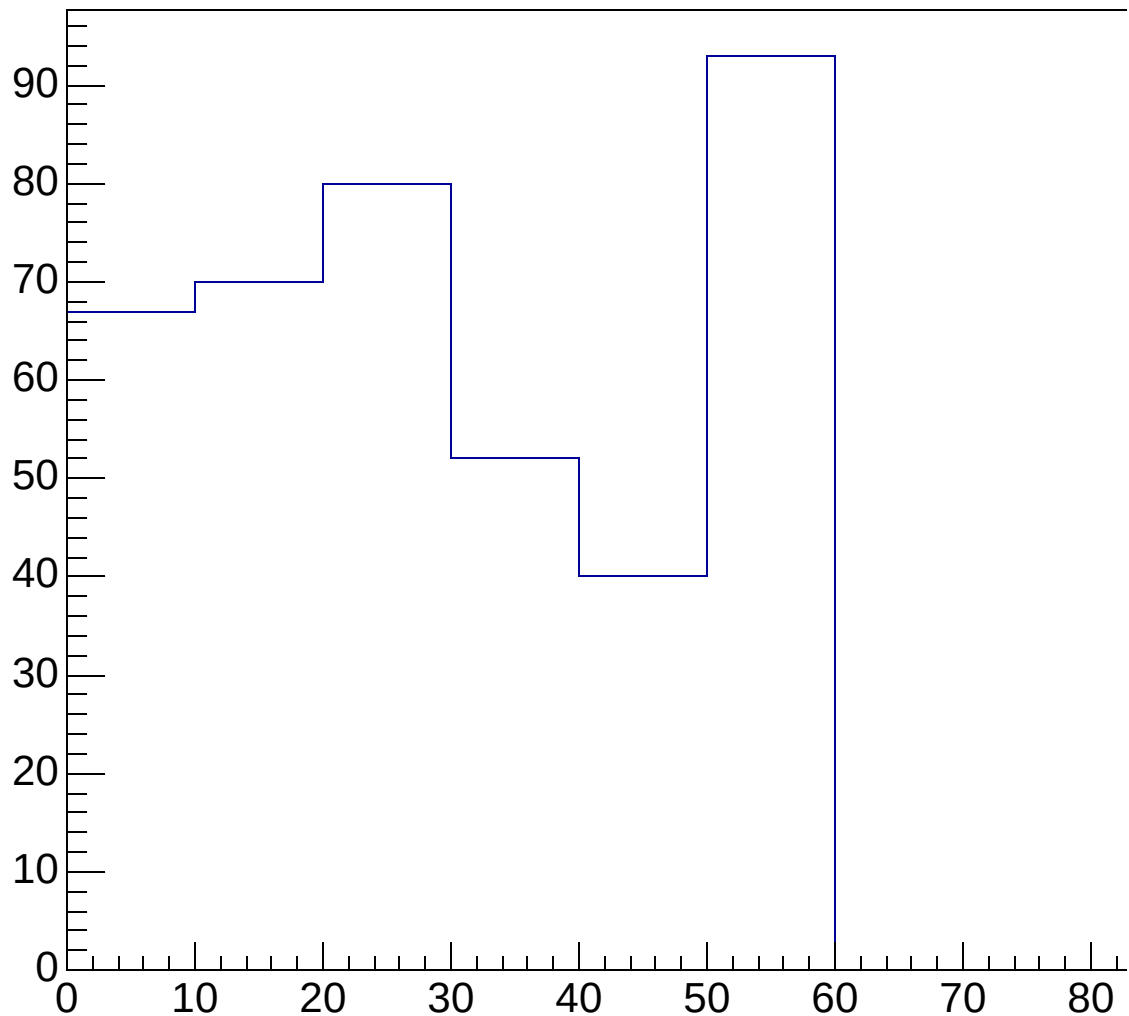
```
In [7]: c1->Draw();
```

Histogram Title



```
In [8]: h1->SetBinContent(1, 20);  
h1->SetBinContent(2, 70);  
h1->SetBinContent(3, 80);  
h1->SetBinContent(4, 52);  
h1->SetBinContent(6, 93);  
h1->SetBinContent(1, 67);  
h1->Draw();  
c1->Draw();
```

Histogram Title



```
In [11]: auto c2 = new TCanvas("c1", "Heat Equation GIF", 800, 600);
auto heat = new TF1("heat", "1.0/sqrt(4*pi*[1]*[0]) * exp(-(x*x)/(4*[1]*[0]))

double alpha = 0.5;
heat->SetParameter(1, alpha);
heat->SetLineColor(kBlue);
heat->SetMinimum(0);
heat->SetMaximum(1);

int nFrames = 30;
for (int i = 1; i <= nFrames; ++i) {
    double t = i * 0.2;
    heat->SetParameter(0, t);
    heat->SetTitle(Form("Time t = %.1f;x;u(x,t)", t));

    heat->Draw();
    c2->Modified();
    c2->Update();

    if (i == 1) {
```

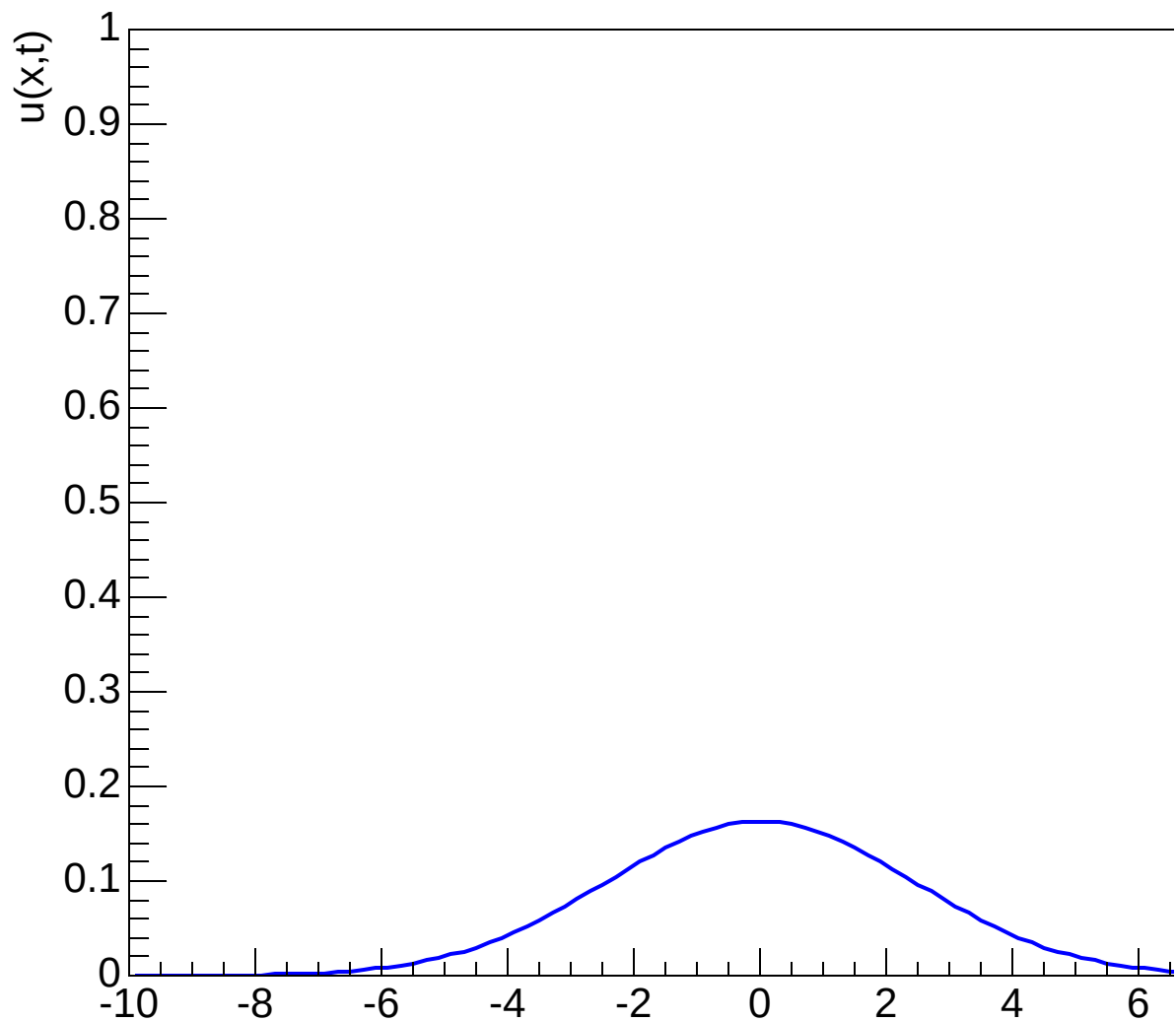


```

// Start the GIF
c2->Print("heat_evolution.gif");
} else if (i == nFrames) {
    // Add the last frame with a '+'
    // Some systems prefer a small delay here
    c2->Print("heat_evolution.gif+");
} else {
    // Append frame with '+'
    c2->Print("heat_evolution.gif+");
}
}

```

Time $t = 6.0$



Warning in <TCanvas::Constructor>: Deleting canvas with same name: c1
 Info in <TCanvas::Print>: gif file heat_evolution.gif has been created

```

In [25]: %%html


```

No description has been provided for this image

```
In [22]: auto c3 = new TCanvas("c3", "3D Heat Evolution", 800, 600);

// [0] = time 't', [1] = alpha
auto heat3D = new TF2("heat3D", "(1.0/(4*pi*[1]*[0])) * exp(-(x*x + y*y)/(4*

double alpha = 0.5;
heat3D->SetParameter(1, alpha);
heat3D->SetTitle("3D Heat Diffusion;X;Y;Temperature");

// Set a professional color palette
gStyle->SetPalette(kBird);

// Fix the Z-axis so the mountain doesn't jump around
heat3D->SetMinimum(0);
heat3D->SetMaximum(0.5);

int nFrames = 400;
for (int i = 1; i <= nFrames; ++i) {
    double t = i * 0.015;
    heat3D->SetParameter(0, t);

    // Draw with color-coded surface
    heat3D->Draw("SURF1");

    c3->Modified();
    c3->Update();

    // Standard ROOT GIF stitching
    if (i == 1) {
        c3->Print("heat_3d_evolution.gif");
    } else {
        c3->Print("heat_3d_evolution.gif+");
    }
}
```

3D Heat Diffusion

Warning in <TCanvas::Constructor>: Deleting canvas with same name: c3
Info in <TCanvas::Print>: gif file heat_3d_evolution.gif has been created

In [8]: `%%html
`

 No description has been provided for this image

In [16]: `// — Cell 1: Read WAV & plot time domain —————

const char* filename = "chimes.wav";
std::ifstream file(filename, std::ios::binary);
if (!file) { std::cout << "Error: Could not open chimes.wav\n"; return; }

file.seekg(44, std::ios::beg); // skip standard WAV header

const int N = 65536; // must be power of 2
std::vector<double> samples;
samples.reserve(N);`

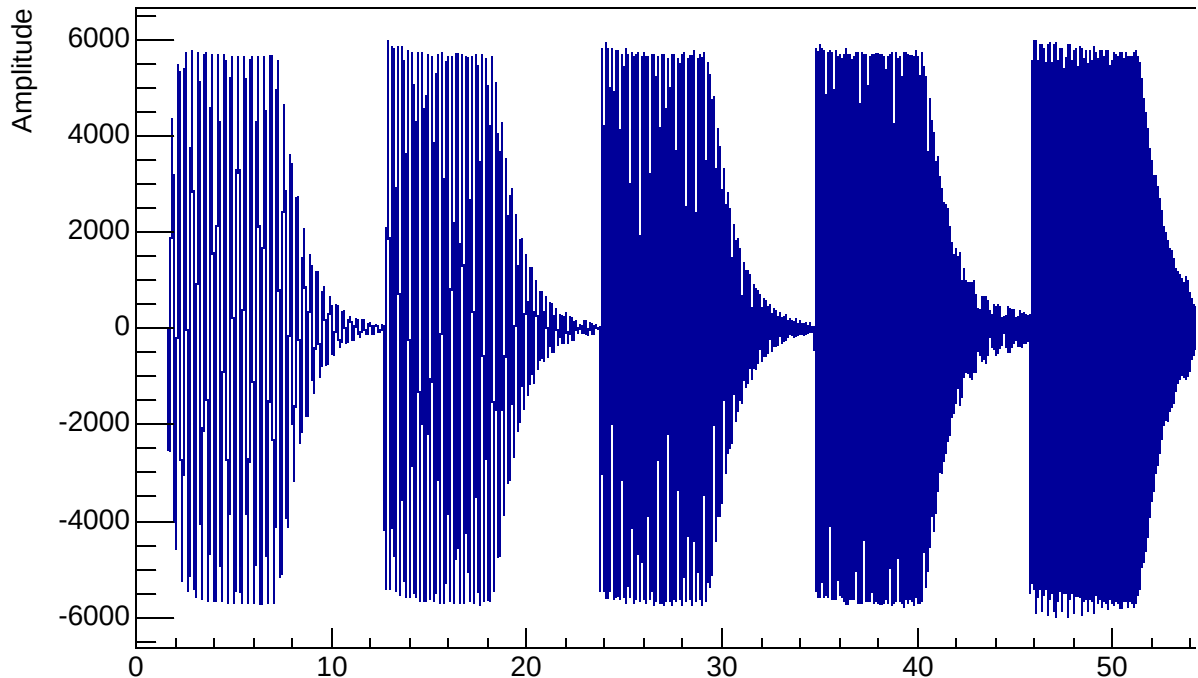
```

short sample_val;
for (int i = 0; i < N && file.read(reinterpret_cast<char*>(&sample_val), sizeof sample_val); ++i)
    samples.push_back(static_cast<double>(sample_val));

auto c_audio = new TCanvas("c_audio", "Raw Audio", 800, 400);
TH1D *h_raw = new TH1D("h_raw", "Original Audio (Time Domain);Sample;Amplitude", 800, 0, N);
for (int i = 0; i < (int)samples.size(); ++i)
    h_raw->SetBinContent(i+1, samples[i]);
h_raw->Draw();
c_audio->Draw();

```

Original Audio (Time Domain)



```

In [17]: // — Cell 2: Forward FFT & frequency spectrum —————
// NOTE: N and samples come from Cell 1 – do NOT re-declare N here

TVirtualFFT *fft = TVirtualFFT::FFT(1, const_cast<int*>(&N), "R2C M K");
fft->SetPoints(samples.data());
fft->Transform();

// Extract complex output into our own arrays so we can modify them safely
std::vector<double> re_arr(N/2 + 1), im_arr(N/2 + 1);
for (int i = 0; i <= N/2; ++i)
    fft->GetPointComplex(i, re_arr[i], im_arr[i]);

auto c_spec = new TCanvas("c_spec", "Frequency Spectrum", 800, 400);
// Label x-axis in Hz assuming 44100 Hz sample rate
double sampleRate = 44100.0;
TH1D *h_spec = new TH1D("h_spec", "Frequency Spectrum;Frequency (Hz);Magnitude", N/2, 0, sampleRate / 2.0);

for (int i = 0; i < N/2; ++i) {
    double mag = std::sqrt(re_arr[i]*re_arr[i] + im_arr[i]*im_arr[i]);
    h_spec->SetBinContent(i+1, mag);
}

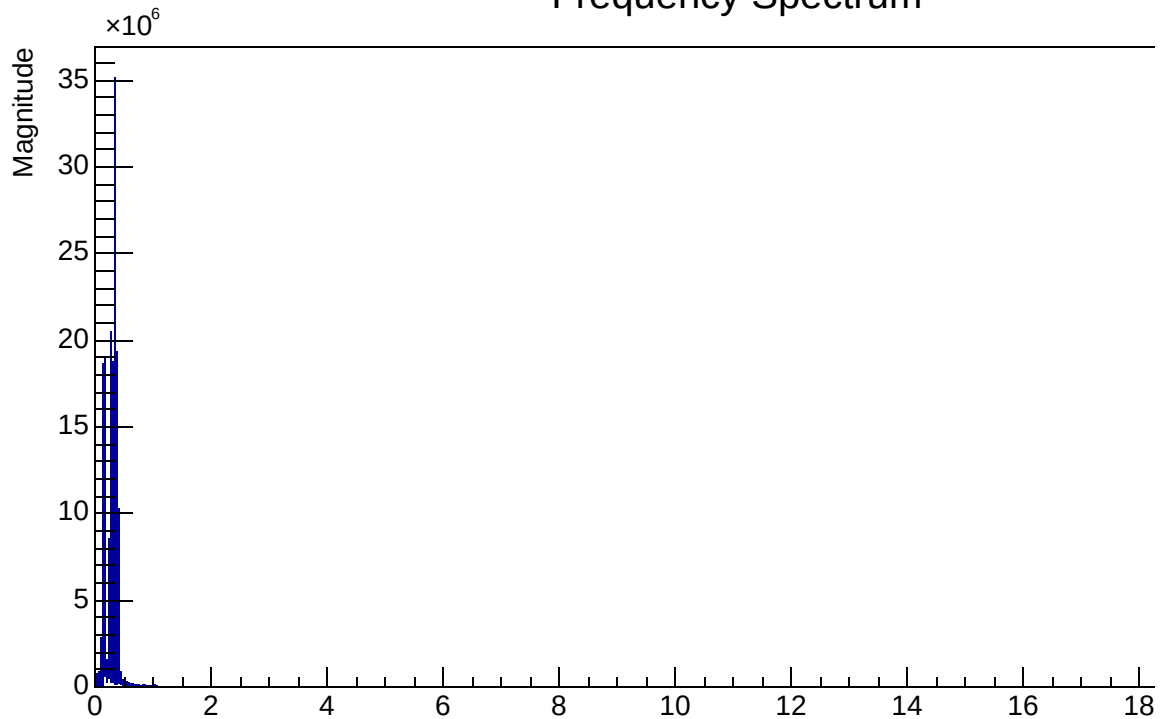
```

```

}
h_spec->Draw();
c_spec->Draw();

```

Frequency Spectrum



```

In [18]: // — Cell 3: Filter + Inverse FFT + save WAV —————
// Set your frequency cutoff range to DISCARD (in Hz):
double freq_lo = 20000.0; // ← lower bound of rejected band (Hz)
double freq_hi = 22000.0; // ← upper bound of rejected band (Hz)
double sampleRate = 44100.0;

```

```

In [19]: // Work on copies of the FFT arrays from Cell 2
std::vector<double> re_filt(re_arr), im_filt(im_arr);

for (int i = 0; i <= N/2; ++i) {
    double freq = i * sampleRate / N; // bin → Hz
    if (freq >= freq_lo && freq <= freq_hi) { // zero out the unwanted band
        re_filt[i] = 0.0;
        im_filt[i] = 0.0;
    }
}

// — Inverse FFT —————
TVirtualFFT *fft_back = TVirtualFFT::FFT(1, const_cast<int*>(&N), "C2R M K")
fft_back->SetPointsComplex(re_filt.data(), im_filt.data());
fft_back->Transform();

// Copy IFFT output into our own array (do NOT pass the internal pointer to
std::vector<double> cleaned(N);
for (int i = 0; i < N; ++i)
    cleaned[i] = fft_back->GetPointReal(i) / N; // FFTW output needs /N not

// — Plot cleaned waveform —————

```

```

auto c_clean = new TCanvas("c_clean", "Cleaned Audio", 800, 400);
TH1D *h_cleaned = new TH1D("h_cleaned",
    "Cleaned Audio (Time Domain);Sample;Amplitude", N, 0, N);
for (int i = 0; i < N; ++i)
    h_cleaned->SetBinContent(i+1, cleaned[i]);
h_cleaned->Draw();
c_clean->Draw();

// — Write output WAV —————
// Find peak for normalisation (preserve original loudness)
double peak = 0;
for (double v : cleaned) if (std::abs(v) > peak) peak = std::abs(v);
double origPeak = 0;
for (double v : samples) if (std::abs(v) > origPeak) origPeak = std::abs(v);
double scale = (peak > 0) ? origPeak / peak : 1.0;

std::vector<short> out_samples(N);
for (int i = 0; i < N; ++i) {
    double val = cleaned[i] * scale;
    // Clamp to 16-bit range
    val = std::max(-32768.0, std::min(32767.0, val));
    out_samples[i] = static_cast<short>(val);
}

// Build a minimal 44-byte PCM WAV header
const char* outfile = "chimes_processed.wav";
std::ofstream wav(outfile, std::ios::binary);

int dataSize = N * sizeof(short);
int chunkSize = 36 + dataSize;
short audioFmt = 1; // PCM
short numChan = 1; // mono
int byteRate = (int)sampleRate * 1 * 2;
short blockAlign = 1 * 2;
short bitsPerSmp = 16;

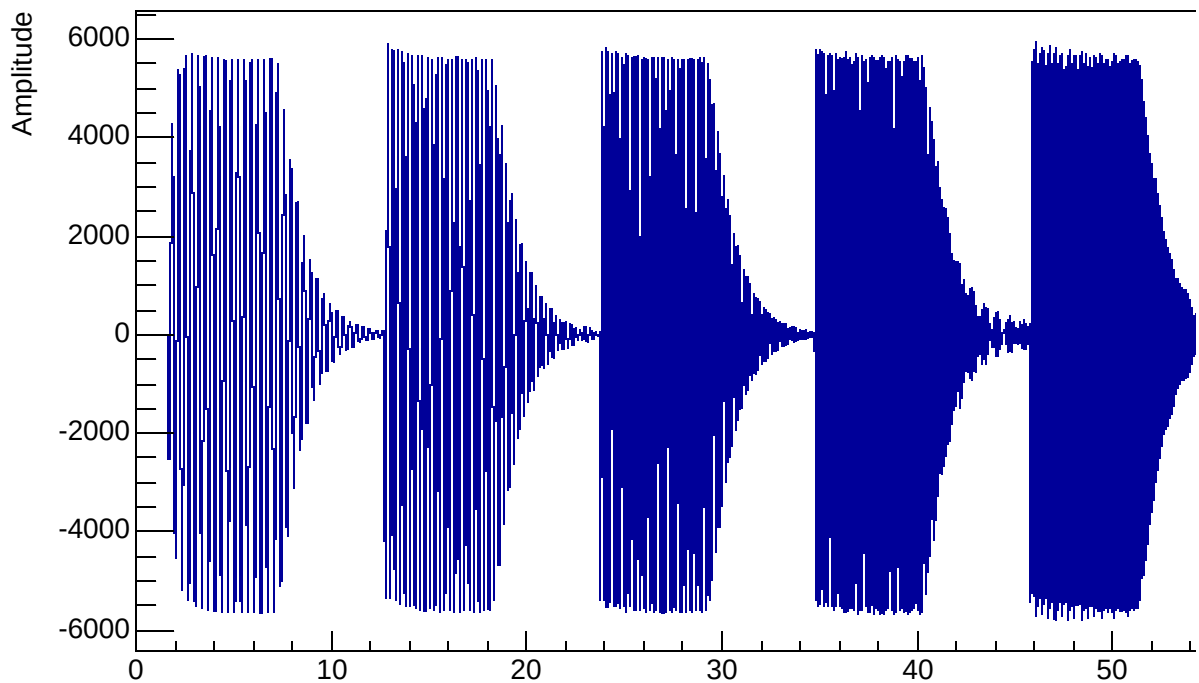
wav.write("RIFF", 4);
wav.write(reinterpret_cast<char*>(&chunkSize), 4);
wav.write("WAVE", 4);
wav.write("fmt ", 4);
int subchunk1 = 16;
wav.write(reinterpret_cast<char*>(&subchunk1), 4);
wav.write(reinterpret_cast<char*>(&audioFmt), 2);
wav.write(reinterpret_cast<char*>(&numChan), 2);
int sr = (int)sampleRate;
wav.write(reinterpret_cast<char*>(&sr), 4);
wav.write(reinterpret_cast<char*>(&byteRate), 4);
wav.write(reinterpret_cast<char*>(&blockAlign), 2);
wav.write(reinterpret_cast<char*>(&bitsPerSmp), 2);
wav.write("data", 4);
wav.write(reinterpret_cast<char*>(&dataSize), 4);
wav.write(reinterpret_cast<char*>(out_samples.data()), dataSize);
wav.close();

std::cout << "Saved: " << outfile

```

```
<< " (" << N << " samples, band " << freq_lo
<< "-" << freq_hi << " Hz removed)\n";
```

Cleaned Audio (Time Domain)



Saved: chimes_processed.wav (65536 samples, band 20000–22000 Hz removed)

Leapfrog Integrator

simulating a rocket flyin off

$$g = GM/r^2$$

To simulate ground we can have a no 0 g at surface at 6.3781kms from center of earth, equal to $g(r_0)$. With $r_o = 6378.1km$, we can have rocket acceleration increasing gradually, slowly equalling $g(r_0)$ for liftoff and then slowly reaching orbital velocity. the rocket must then tilt slowly in x-direction for the usual path and then see how that goes.

all these params for the rocket thrust equation

$$\mathbf{F}_{\text{thrust}} = \dot{m}v_e + (p_e - p_a)A_e$$

- $\dot{m}$ (mass_flow_rate): The mass of propellant (fuel + oxidizer) expelled per unit time ($\frac{kg}{s}$).
- v_e (exit_velocity): The velocity of the exhaust gas relative to the nozzle exit.

- $\dot{m}v_e$ (Momentum Thrust): The primary thrust component, calculated as mass flow rate multiplied by the exhaust velocity.
- $(p_e - p_a)A_e$ (Pressure Thrust): The additional thrust (or drag) created when the exit pressure (p_e : exit_pa) differs from the ambient, or atmospheric, pressure (p_a : atm) acting across the nozzle exit area (A_e : nozzle_exit_area). [2, 3, 4, 5]

we launching ISRO's LVM3

In [1]: `#include "schrodingerlib.cpp"`

In [2]: `const int n = 10000;
std::vector<double> x(n);
std::vector<double> v(n);
std::vector<double> t(n);
// to be evaluated from the field interp at each step, assuming constant for
std::vector<double> a(n);
std::vector<double> atm_pad(n);
std::vector<double> mass_pad(n);
const double G = 6.67e-11; // m3 kg-1 s-2
const double M = 5.9722e+24; // kg
const double R = 6378100.0f; // m`

In [3]: `v.at(0) = 0.0f;
x.at(0) = 0.0f;
t.at(0) = 0.0f;

double dt = 0.1f;

// we are on earth, launching from equator (say, french guyana)

double g = (G * M) / (R * R); // m s-2
double thrust = 0;

// double m_rocket = 640000; // 640 Tonnes
// double mass_flow_rate = 571.4; // kg s-1 -> constant for a stage
// double exit_velocity = 2550; // m s-1 -> constant for a stage
// double exit_pa = 52000; // pascals -> constant for a stage
// double nozzle_exit_area = 0.8; // m2 -> constant for a stage
// double atm = 101325.0 // pascals -> decreases exponentially

double mass_flow_rate; // kg s-1 -> constant for a stage
double exit_velocity; // m s-1 -> constant for a stage
double exit_pa; // pascals -> constant for a stage
double nozzle_exit_area; // m2 -> constant for a stage
bool s200_jettisoned = false;
bool l110_jettisoned = false;
bool plf_jettisoned = false;
bool plf_js_log = false;
double prop_s200 = 410000;
double prop_l110 = 116000;`


```

double prop_c25 = 28000;
bool liftoff = false;

double thrust_accn = 0;
double INITIAL_VELOCITY = 0;
double m_rocket = 640000; // 640 Tonnes

double atm = get_ambient_pressure(0, R);
double drag_force;
double drag_accn;

atm_pad.at(0) = atm;

// initialization -> half integer velocity
get_lvm3_stage_properties(0, 0.0, m_rocket, mass_flow_rate, exit_velocity, e
    s200_jettisoned, l110_jettisoned, plf_jettisoned, prop_s200, prop_l110,
    thrust = (mass_flow_rate * exit_velocity) + (exit_pa - atm) * nozzle_exit_a
    thrust_accn = thrust / m_rocket;

double v_half = INITIAL_VELOCITY + a.at(0) * (dt / 2.0f);
v.at(0) = v_half;
// 'v' array will now track v-values at 1/2, 3/2, 5/2, ... time steps

```

```

In [4]: for(int i = 0; i < n - 1; i++) {
    // time evolution leapfrogging (v goes 1/2 integers, and x goes integers)
    // The normal Euler update, using an abnormal derivate (v_{i + half}) in
    x.at(i + 1) = x.at(i) + v_half * dt;

    if (v_half > 1.0) liftoff = true;

    // landed / crashed / whatever
    if (liftoff && x.at(i + 1) <= 0.0) {
        x.at(i + 1) = 0.0;
        v_half = 0.0;
        for (int j = i + 1; j < n; j++) {
            x.at(j) = 0.0;
            v.at(j) = 0.0;
            a.at(j) = 0.0;
            t.at(j) = t.at(j - 1) + dt;
            atm_pad.at(j) = get_ambient_pressure(0.0, R);
            mass_pad.at(j) = m_rocket;
        }
        break;
    } else if (x.at(i + 1) <= 0.0) {

        // clamp for x,v when thrust cant quite overcome gravity
        x.at(i + 1) = 0.0;
        v_half = 0.0;
    }

    // as soon as position changes, we need to update the new 'g'

```

```

g = (G * M) / ((x.at(i + 1) + R) * (x.at(i + 1) + R));

// whats the pressure at this height
atm = get_ambient_pressure(x.at(i + 1), R);

// refreshing the thrust related components
// loose some mass (in form of fuel) -> pehle mass loose hoga tabhi accn
get_lvm3_stage_properties(x.at(i + 1), dt, m_rocket, mass_flow_rate,
    exit_velocity, exit_pa, nozzle_exit_area,
    s200_jettisoned, l110_jettisoned, plf_jettisoned, prop_s200, prop_l110);

// the standard rocket thrust equation
thrust = (mass_flow_rate * exit_velocity) + (exit_pa - atm) * nozzle_exit_area;
thrust_accn = (m_rocket > 1.0) ? thrust / m_rocket : 0.0;

if (thrust_accn <= g && (x.at(i + 1)) <= 1e-3) {
    // Rocket does not have enough thrust to lift off yet; remain dead s
    a.at(i + 1) = 0.0;
    // Reset velocity offset to zero
    v_half = 0.0;
    v.at(i + 1) = 0.0;
} else {
    // Flying or cleared pad: compute acceleration
    a.at(i + 1) = thrust_accn - g;

    drag_force = get_lvm3_drag_force(x.at(i + 1), v_half, atm);
    drag_accn = drag_force / m_rocket;

    // Drag always works against the direction of movement
    if (v_half > 0) {
        a.at(i + 1) -= drag_accn;
    } else {
        a.at(i + 1) += drag_accn;
    }

    // Advance velocity from half-step (i + 0.5) to next half-step (i + 1)
    v_half = v_half + a.at(i + 1) * dt;

    // syncing v array to integer indices for plotting later
    v.at(i + 1) = v_half - a.at(i + 1) * (dt / 2.0);
}

t.at(i + 1) = t.at(i) + dt;

// tracking ts for plot later
atm_pad.at(i + 1) = atm;
mass_pad.at(i) = m_rocket;
}

```

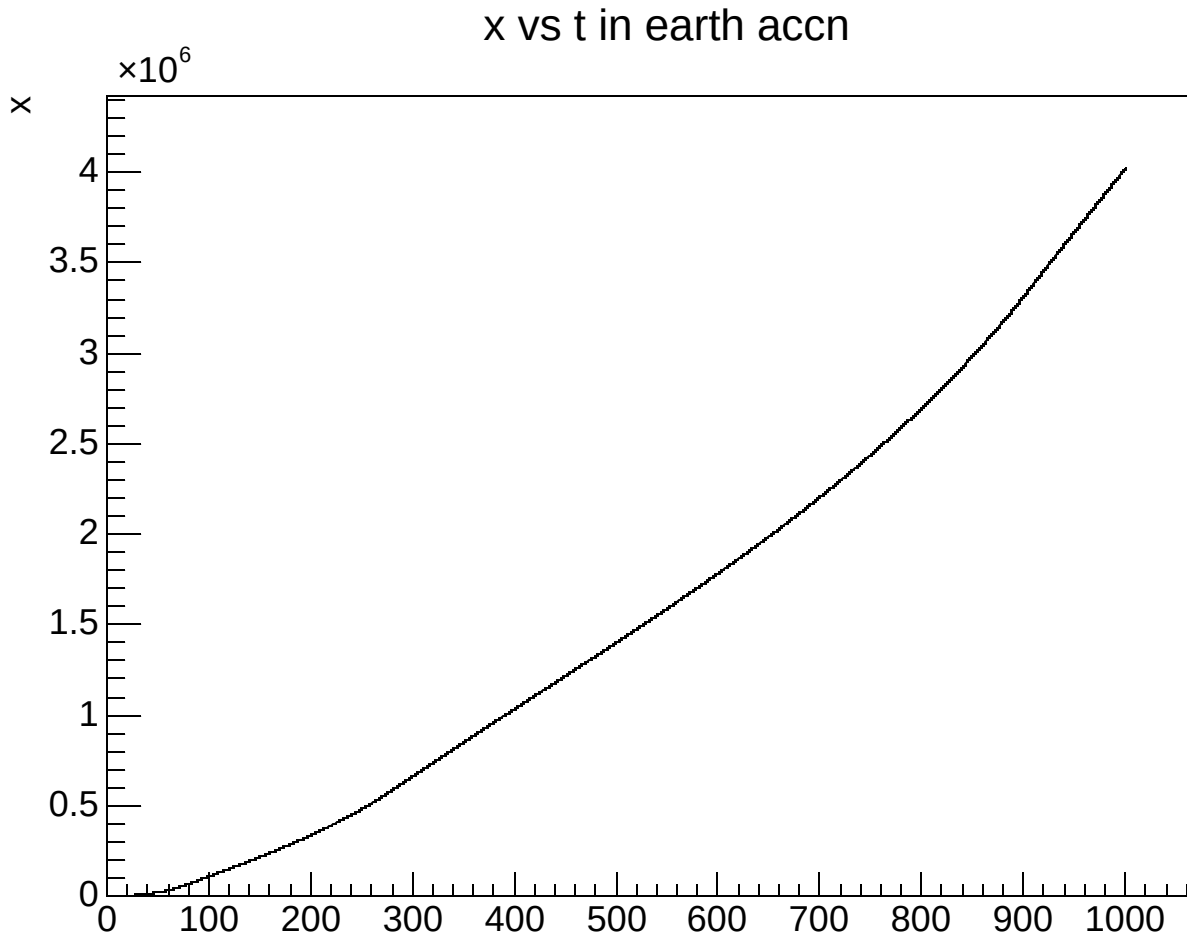
s200 jettisoned, 167.954tonnes, 57.6225kms,
plf jettisoned, 150.512tonnes, 115.015kms,
l110 jettisoned, 35.4027tonnes, 579.368kms,

```

In [5]: TGraph* gr = new TGraph(n, t.data(), x.data());
TCanvas* c1 = new TCanvas("c1", "Position vs time");
gr->Draw("ALP");

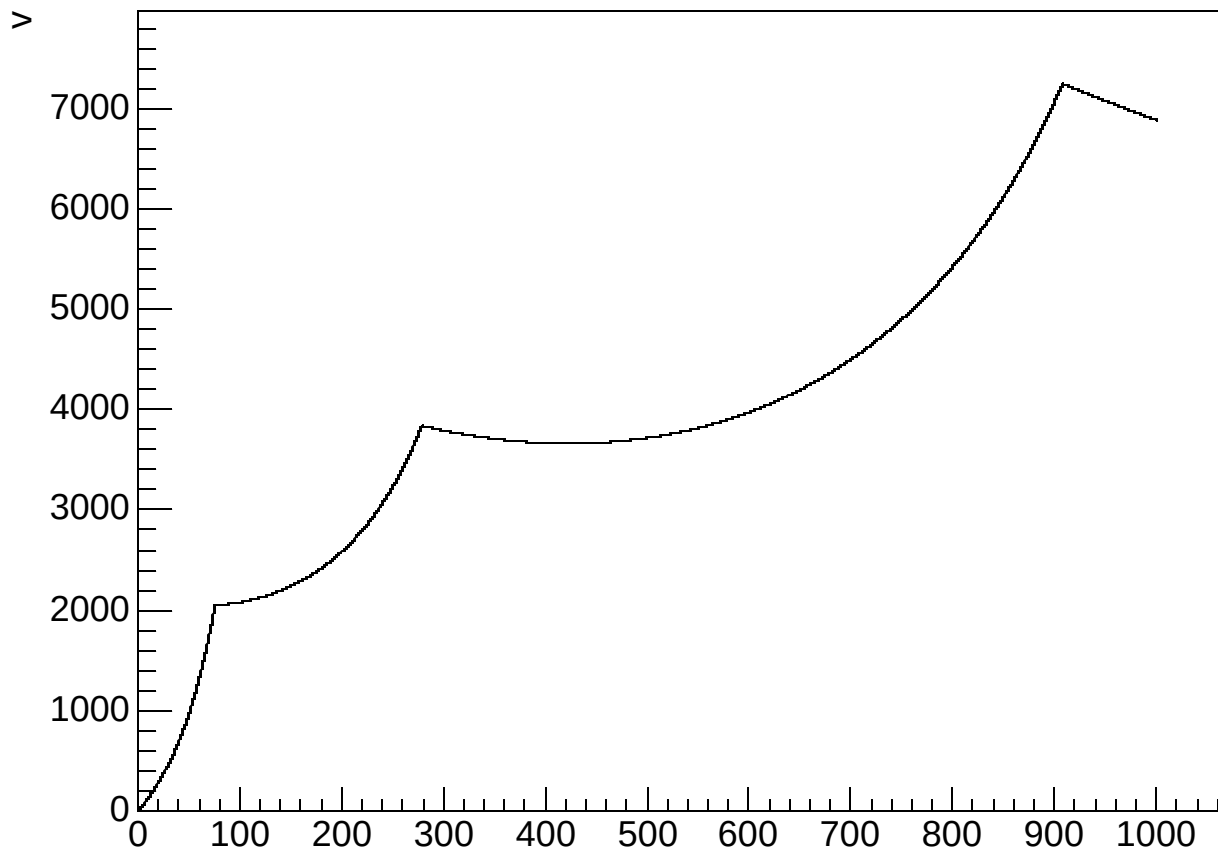
```

```
gr->SetTitle("x vs t in earth accn;t;x");
c1->Draw();
```

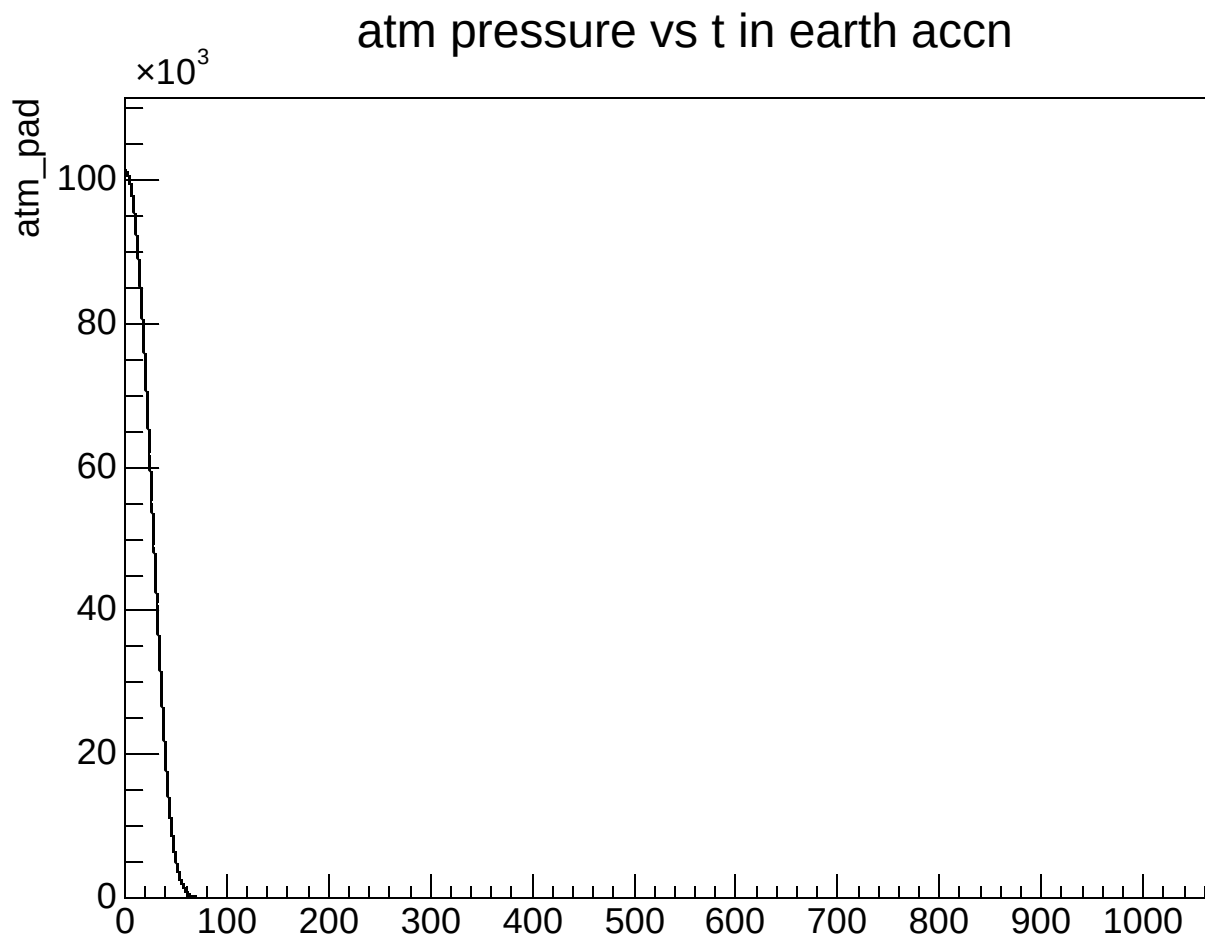


```
In [6]: TGraph* gr2 = new TGraph(n, t.data(), v.data());
TCanvas* c2 = new TCanvas("c2", "Velocity vs time (half shifted to right)");
gr2->Draw("ALP");
gr2->SetTitle("v_half vs t + 1/2 in earth accn;t;v");
c2->Draw();
```

v_half vs t + 1/2 in earth accn

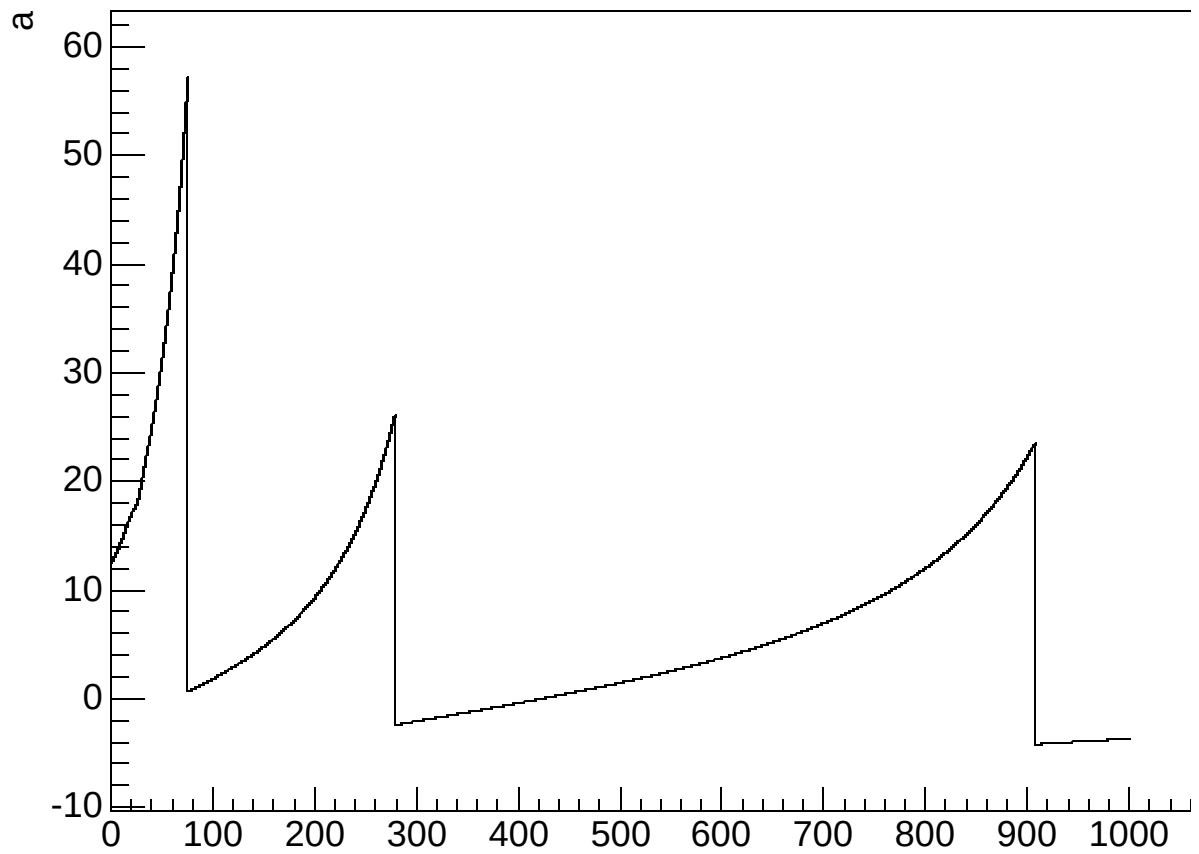


```
In [7]: TGraph* gr3 = new TGraph(n, t.data(), atm_pad.data());
TCanvas* c3 = new TCanvas("c3", "atm pressure vs time");
gr3->Draw("ALP");
gr3->SetTitle("atm pressure vs t in earth accn;t;atm_pad");
c3->Draw();
```



```
In [8]: TGraph* gr4 = new TGraph(n, t.data(), a.data());
TCanvas* c4 = new TCanvas("c4", "a vs time");
gr4->Draw("ALP");
gr4->SetTitle("a vs t in earth accn;t;a");
c4->Draw();
```

a vs t in earth accn



```
In [9]: TGraph* gr5 = new TGraph(n, t.data(), mass_pad.data());
TCanvas* c5 = new TCanvas("c5", "mass of rocket vs time");
gr5->Draw("ALP");
gr5->SetTitle("mass of rocket vs time in earth accn;t;m_rocket");
c5->Draw();
```

mass of rocket vs time in earth accn

